T03

Mobile Automation Testing với Appium 2 và Applitools Eyes

Danh sách thành viên:
23120027 - Nguyễn Hải Đăng
23120031 - Nguyễn Phú Dinh
23120034 - Ứng Thị Thuỳ Dương

2026 AI-First · CS423/CSC15003





DEMO: Appium 2 + UiAutomator2 (truyền thống) · Applitools Eyes (AI visual)

Tại sao kiểm thử mobile khó khăn?

- **Phân mảnh thiết bị:** phiên bản Android/iOS, kích thước và mật độ màn hình.
- **Giao diện thay đổi theo trạng thái:** bàn phím, thông báo, quyền, animation, network.
- **Regression testing:** phải kiểm thử lại sau khi app được cập nhật.





Hai khía cạnh của kiểm thử giao diện

Hai câu hỏi của kiểm thử giao diện

	Functional	Visual
Hỏi	App **làm** đúng không?	App **trông** đúng không?
Chuẩn so	Giá trị kỳ vọng trong code	Ảnh baseline đã duyệt
Công cụ	Appium	Applitools Eyes

Vì sao cần cả hai

- Functional pass **không** chứng minh giao diện đúng
- Visual pass **không** chứng minh dữ liệu đúng
- Chạy ở đâu: emulator (rẻ, lặp lại được) hay thiết bị thật / device farm (đúng người dùng, đắt hơn)

Khảo sát công cụ — truyền thống và AI-augmented

Truyền thống

Appium 2 — chuẩn WebDriver, đứng ngoài app, Android + iOS, không cần mã nguồn. ✅ **chọn làm lớp functional**

Espresso — chạy trong app, nhanh, chỉ Android, viết Kotlin. ✗

Detox — cho React Native, tự đồng bộ animation/network, cần build riêng và Node 22, chỉ JS. **Đã chạy thử cùng kịch bản: 3/3 lượt pass.** Hợp khi đội RN chỉ cần CI nhanh.

AI-augmented

Applitools Eyes — so ảnh với baseline bằng Visual AI, có ignore/layout region và dashboard duyệt. ✅ **chọn làm lớp visual**

Maestro + AI assertion — flow YAML, `assertWithAI` bằng LLM, không có baseline, không lịch sử duyệt. Dự phòng.

Vì sao ghép Appium + Eyes

Eyes gắn vào **cùng một session** Appium: assert chức năng và checkpoint visual chạy trên cùng luồng, cùng APK người dùng cài, chạy được device farm. Detox không gắn được visual AI theo cách đó.

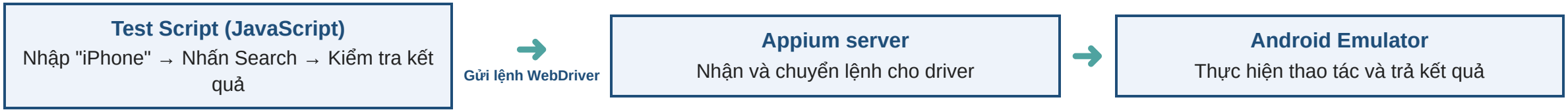
Appium là gì?

Appium là framework mã nguồn mở dùng để tự động hóa kiểm thử giao diện ứng dụng mobile thông qua các test script.

Có thể làm gì?

- Tự động mở ứng dụng, nhập dữ liệu, nhấn nút.
- Tìm các thành phần giao diện và kiểm tra kết quả.
- Hỗ trợ Android và iOS thông qua các driver tương ứng.

Hoạt động thế nào?



Bắt đầu một Appium Test — Session & Capabilities

```
// src/appium/config/session.js
const driver = await remote({
  hostname: "127.0.0.1", port: 4723, path: "/",        // Appium 2 dùng "/"; Appium 1 dùng "/wd/hub"
  capabilities: {
    platformName: "Android",
    "appium:automationName": "UiAutomator2",           // driver
    "appium:app": ".../app-debug.apk",                  // cài APK này khi tạo session
    "appium:appPackage": "com.eshop.sut",
    "appium:noReset": false,                            // xoá dữ liệu app → mỗi test bắt đầu sạch (~25 s)
  },
});
```

- **Session:** Phiên làm việc giữa test script và Appium Server để điều khiển ứng dụng trên thiết bị.
- **Capabilities:** Các thông tin cấu hình cho Appium biết cần sử dụng nền tảng, driver và ứng dụng nào.

Create Session
Kết nối và chuẩn bị ứng dụng

→

Execute Test
Tìm phần tử, thao tác, kiểm tra kết quả

→

End Session
Giải phóng tài nguyên

Locator — Appium tìm phần tử giao diện như thế nào?

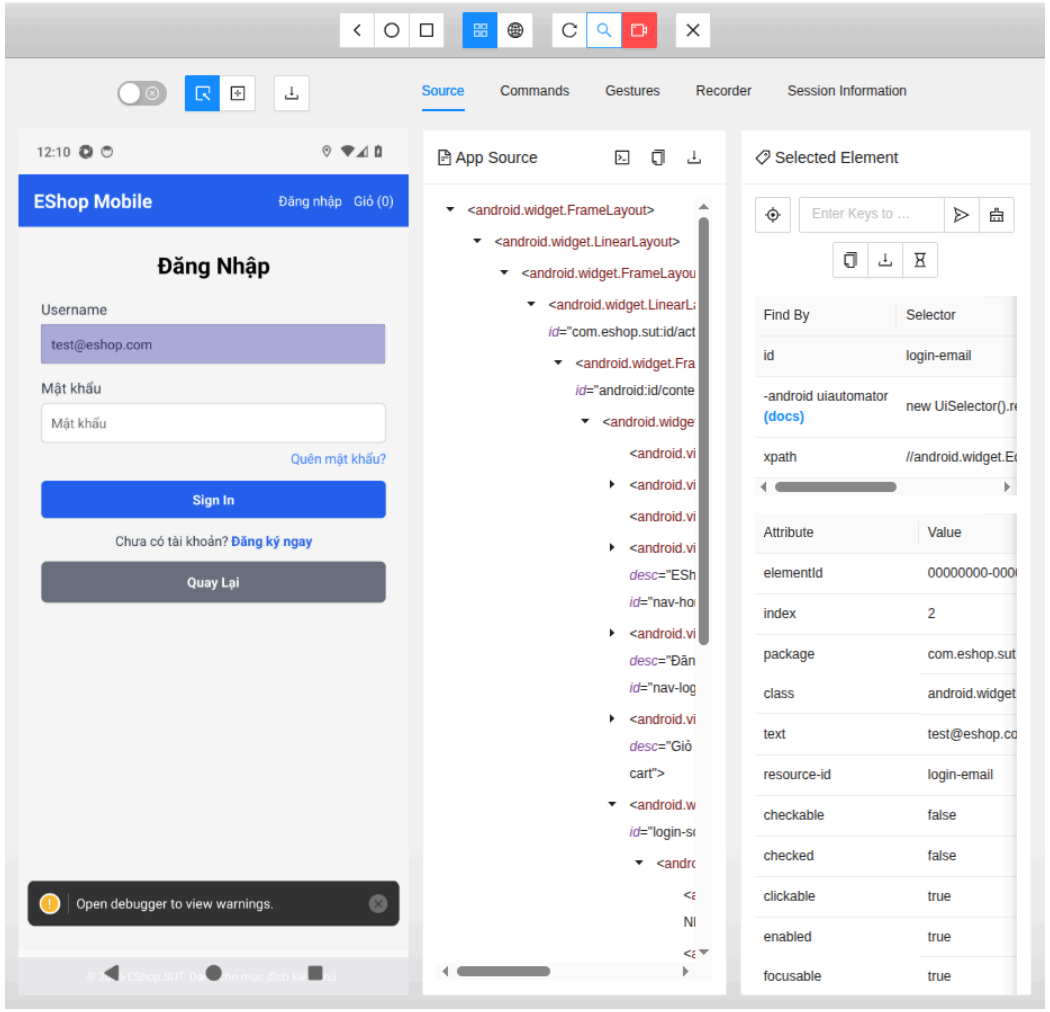
Locator: là cách xác định một phần tử trên giao diện ứng dụng để Appium có thể tương tác với nó.
Ví dụ: Để Appium nhập email, trước tiên phải tìm đúng ô nhập email.

```
<android.widget.EditText resource-id="login-email" ... />
```

Ưu tiên	Chiến lược	Cú pháp WebdriverIO	Ghi chú
1	`resource-id`	`android=new UiSelector().resourceId("login-email")`	Ổn định, do dev đặt
2	accessibility id	`~Đăng nhập`	Từ `accessibilityLabel`
3	text	`android=new UiSelector().text("Tìm")`	Gãy khi đổi chữ / ngôn ngữ
4	XPath	`//android.widget.EditText[@resource-id="login-email"]`	Chậm, gãy khi đổi bố cục

Best practice: Ưu tiên Locator ổn định, ít phụ thuộc vào vị trí và cấu trúc giao diện để giảm chi phí bảo trì test.

Appium Inspector — Khám phá giao diện và tìm Locator



Appium Inspector là gì?

Appium Inspector là công cụ hỗ trợ quan sát cấu trúc giao diện, xem thuộc tính phần tử và thử các thao tác trên ứng dụng.

3 tính năng chính:

- **Inspect UI Elements:** Xem cây giao diện và các thuộc tính của phần tử.
- **Find & Test Locators:** Tìm phần tử bằng locator và thử các thao tác như Tap, Send Keys.
- **Record Actions:** Ghi lại thao tác và hỗ trợ tạo code mẫu để xây dựng test script.

=> Appium Inspector giúp chúng ta **tìm Locator trước khi viết test script**.

Viết một Appium Test — Locator, Action & Assertion

Kịch bản — Verify Cart Total: thêm *iPhone 15 Pro Max* (30 000 000 ₫) vào giỏ 2 lần → tổng tạm tính phải là **60 000 000 ₫**.

1 · Locator
Tìm nút "Thêm vào giỏ" (`product-add-1`)

→

2 · Action
Bấm 2 lần, mở giỏ hàng

→

3 · Wait
Chờ dòng tổng tiền hiện ra

→

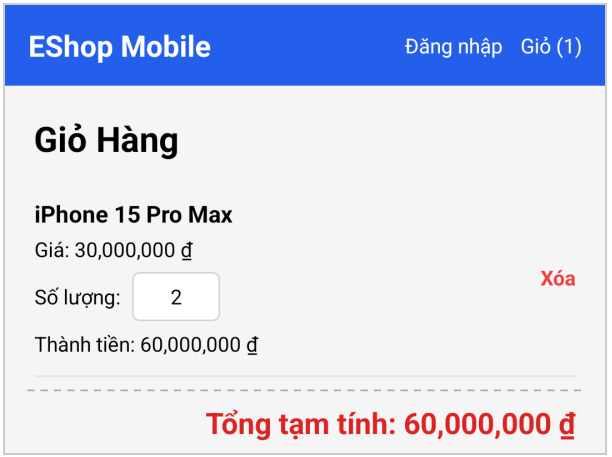
4 · Assertion
Tổng = 60 000 000 ₫?

→

Test Result
PASS / FAIL dựa trên Assertion

```
const byId = (id) => driver.$(`android=new UiSelector().resourceId("${id}")`);
const addBtn = await byId("product-add-1");          // 1 Locator
for (let i = 0; i < 2; i++) {                        // 2 Action: thêm 2 lần
  await addBtn.click();
  await (await byId("android:id/button1")).click();  // đóng hộp thoại "Thành công"
}
await (await byId("nav-cart")).click();              // mở giỏ hàng
const total = await byId("cart-total");
await total.waitForDisplayed({ timeout: 10_000 });   // 3 Wait: chờ điều kiện
assert.equal((await total.getText()).replace(/\D/g, ""), "60000000");  // 4 Assertion
```

Best practice: Explicit Wait — chờ điều kiện cụ thể thay vì dùng thời gian chờ cố định. Bộ test thật gom các bước này vào Page Object (`home.addToCart(1, 2)`).



Màn giỏ hàng EShop lúc test chạy trên emulator (23/10/2026).

Test này pass vì tổng tiền đúng. Nhưng nếu dòng tổng lệch sang trái và nút thanh toán đổi màu, Appium có phát hiện được không? → Applitools Eyes

Visual testing và Applitools Eyes

- **Checkpoint** = ảnh chụp tại một bước; **baseline** = ảnh đã được người duyệt. Visual AI so cấu trúc và bỏ nhiễu render, không so từng pixel
- **Match level** (Strict, Layout…) và **ignore / layout region** cho vùng động (đồng hồ, banner)
- Kết quả **Passed / Unresolved** → người **Accept** (thành baseline mới) hoặc **Reject** (bug). Công cụ chỉ báo "khác", con người quyết định

Gắn vào Appium: cùng session, chụp sau khi assert xong

```
const eyes = createEyes();      // null nếu không có khoá → chỉ còn functional
await openEyes(eyes, driver, "Cart add and total");
await checkpointAs(eyes, "cartFilled", "Cart - 1 item with total");
await closeEyes(eyes);          // in "Eyes: Passed | … | link dashboard"
```

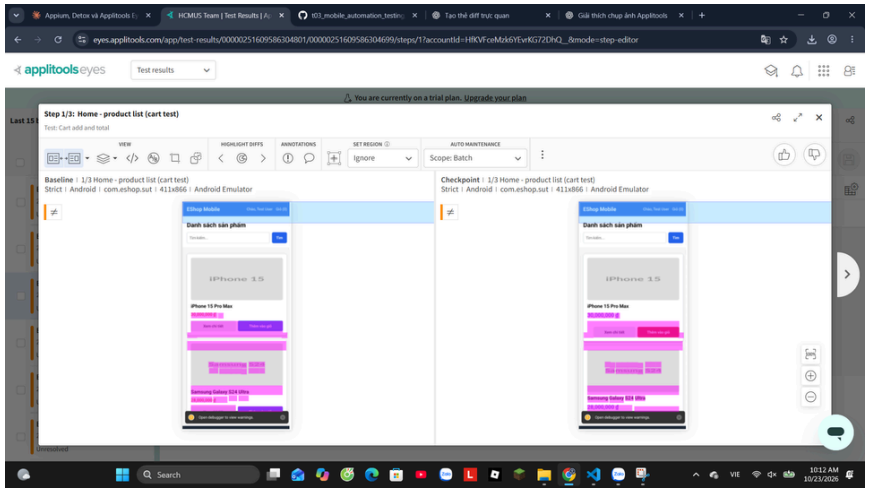
Eyes là **lớp cộng thêm**: hết trial hay mất mạng thì 8 kiểm tra chức năng vẫn chạy đủ

Luồng làm việc: chạy base → Accept baseline trên dashboard → chạy bản mới → triage từng checkpoint

Ba bản giao diện, hai lớp kiểm thử

Bản	Thay đổi	Appium	Eyes
base	bản gốc (baseline)	8/8 pass	Passed
V1	đổi **ảnh** sản phẩm 1, bố cục giữ nguyên	8/8 pass	Unresolved 5/10
V2	đổi **màu** giá, màu nút, **lệch** nút và dòng tổng tiền	8/8 pass	Unresolved 6/10

- **V1** = thay đổi hợp lệ → Eyes báo "thừa" nếu coi là lỗi → **Accept**, hoặc đặt vùng Layout/Ignore lên ảnh
- **V2** = regression thật mà functional **không thấy** (chữ, số, testID đều y nguyên) → **Reject**
- Chi phí bảo trì: đổi ảnh rẻ cho Appium, tốn công duyệt cho Eyes; đổi màu/bố cục thì ngược lại



V2 trên dashboard: baseline (trái) – checkpoint (phải), vùng khác biệt được tô.

Clip demo: **[link + mốc thời gian]**

Giới hạn của hai công cụ

Appium

- **Không tự đánh giá giao diện** – Functional tests có thể pass dù UI hiển thị sai.
- **Nhạy cảm với thay đổi UI** – Test scripts có thể cần bảo trì khi locators thay đổi.
- **Phụ thuộc môi trường thực thi** – Thiết bị, animation và thời gian tải có thể ảnh hưởng tính ổn định.

Applitools Eyes

- **Không kiểm tra đầy đủ chức năng** – UI đúng không đảm bảo logic hoặc dữ liệu đúng.
- **Cần phân biệt thay đổi và lỗi** – Không phải visual difference nào cũng là bug.
- **Cần quản lý baseline** – Những thay đổi thiết kế hợp lệ phải được review và cập nhật.

Visual Diff Triage Workshop

Q&A